

クワッドコプタのロータ推力推定と 非線形モデル予測制御を統合した耐故障制御

Fault-Tolerant Control of Quadcopter Integrating Rotor Thrust Estimation and Nonlinear Model Predictive Control

京都大学 ○ 室木 大典, 野呂 拓臣, 河村 昌典, 島 岳也, 大塚 敏之

Daisuke Muroki, Takumi Noro, Masanori Kawamura, Takeya Shima and Toshiyuki Ohtsuka
Kyoto University

Abstract This study presents a method for fault diagnosis and fault-tolerant control of a quadcopter. The thrust of each rotor is estimated by solving a least-squares problem that considers the relationship between the acceleration, angular velocity, and thrust of the quadcopter. The ratio of the estimated thrust to the control input is then used for fault diagnosis, which is integrated into nonlinear model predictive control. This approach enables a single controller to handle a wide range of faults regardless of the degree of rotor effectiveness loss or the position and number of failed rotors. Finally, the effectiveness of the proposed method is verified through several simulations that assume an actual quadcopter.

1 はじめに

近年、ドローンと呼ばれる無人航空機 (UAV: Unmanned Aerial Vehicle), その中でも特に 3 つ以上のロータを持つマルチコプタの活用が, 農業や物流, インフラの点検, 災害の救援などといった幅広い産業分野で進んでいる [1]. マルチコプタを実社会で活用する上での課題として, 墜落や衝突などによる事故の危険性が挙げられる. よって, 事故を防止するための安全対策が必要不可欠である. マルチコプタの事故原因として, 機体本体の故障が挙げられるが, その中でもロータや駆動モータなどによって構成される推進系の故障は, マルチコプタが制御不能となる状態を招き, 重大事故に繋がる. しかしながら, マルチコプタの推進系は冗長化を実現することが難しい [2]. よって, 推進系の故障に対応した耐故障制御 (Fault-Tolerant Control: FTC) 手法の実現が必須である. そこで本研究では, マルチコプタ, 特に 4 枚のロータを持つクワッドコプタの推進系における様々な故障に対して有効性のある耐故障制御手法を提案することを目的とする.

クワッドコプタの耐故障制御については, 多くの手法が提案されている. 一般的に耐故障制御の手法は大きく 2 つに分けられ, パッシブな耐故障制御 (Passive Fault-Tolerant Control: PFTC) とアクティブな耐故障制御 (Active Fault-Tolerant Control: AFTC) がある. 前者は, ロバスト制御の一種であり, 故障をモデルの外乱や不確実性とみなすことで, 故障時でも飛行を継続させる. パッシブな耐故障制御は, ロータが少し欠けると

いった軽度な故障 (不具合) に対処できるが, 許容できるモデルの不確かさには限度があるため, ロータの有効性が完全に損失するといった重度な故障には対応できない. 一方, アクティブな耐故障制御は, 故障診断 (Fault Detection and Diagnosis: FDD) を行っただけで, その情報に基づいて制御を行う. アクティブな耐故障制御においては, 故障情報に基づいてヨー方向 (上から機体を見たときの回転方向) の制御を犠牲にすることで, 軽度な故障のみならず 1 つのロータが完全に停止するような重度な故障においても, 機体の飛行を継続できることが示されている [3-6]. しかし, これらの先行研究の手法においては, 制御器の切替が必要で, 故障条件が限定されていたり, 故障を前提として予めヨー方向制御が犠牲になったりしているという問題がある. また, モデルを線形化しているため, 平衡点以外で制御性能が悪化する. モデルを線形化せずに扱える非線形モデル予測制御 (Nonlinear Model Predictive Control: NMPC) により耐故障制御を行っている例もある [6, 7] が, これらの研究においては, 故障診断は行われていない. つまり, 故障は必ず発生し, 即座に診断できるという前提で制御系を構築している.

以上の先行研究を踏まえたうえで, 本研究の貢献を次に示す. まず, 推定推力を NMPC の制御入力で割ることによる, クワッドコプタの故障を診断する新しい手法を提案する. 故障診断において, 機体の位置情報は不要なので GPS やモーションキャプチャなどの外部計測には依存しない. また, 機体の加速度・角加速度センサで大きく生じやすいノイズの影響を軽減し, 診断の誤差を

小さくする．次に前述した故障診断手法を NMPC と統合することで，線形化を行わず，制約条件を考慮したアクティブな耐故障制御を実現する．提案する耐故障制御においては，制御器の切り替えなしで無数のパターンのロータ故障を扱える．つまり，ロータの有効性損失の程度や故障の発生するロータの位置や数を限定しない．一般的なロータ故障として，制御入力にかかわらずロータの推力が一定値に固定される場合や，推力が制御入力よりも大きくなってしまう場合も考えられるが [8]，そのような状況でも提案手法は有効である．また，NMPC における評価関数の重みを故障状態に応じて変化させることで，故障が重度な場合のみヨー方向回転の制御を犠牲にし，水平方向の位置制御よりも高度の維持を優先する．不具合の無い正常状態や故障が軽度な場合はヨー方向の制御を放棄しない．

本稿の構成は以下のとおりである．第 2 節では，クォータニオンを用いたクワッドコプタの状態方程式について述べる．第 3 節では，ガウスニュートン法による推力推定の手法について述べる．第 4 節では，故障診断と NMPC を統合した耐故障制御の手法について述べる．第 5 節では，実機を想定するモデルにおいて，耐故障制御のシミュレーションを行う．最後に，第 6 節では本稿のまとめと今後の展望を述べる．

2 クワッドコプタのモデリング

クワッドコプタの状態方程式を文献 [9,10] を参考にして導く．本研究では，オイラー角によるジンバルロックの問題を回避するために，クワッドコプタの姿勢表現にクォータニオンを用いる．クォータニオンと共役クォータニオンの成分を便宜上，それぞれ式 (1) のようにベクトルで表す．

$$\begin{aligned} \mathbf{q} &= (q_0 \quad q_1 \quad q_2 \quad q_3)^\top \\ \mathbf{q}^\dagger &= (q_0 \quad -q_1 \quad -q_2 \quad -q_3)^\top \end{aligned} \quad (1)$$

まず，図 1 に示すように各ロータに 1 から 4 の番号を振り，各座標系を設定する． e^i , e^b はそれぞれ慣性座標系と，クワッドコプタの重心を原点とする物体座標系の座標軸を表す．このとき，慣性座標系における機体の重心位置を $\xi = (x \ y \ z)^\top \in \mathbb{R}^3$ ，物体座標系の座標軸 e_i^b ($i = 1, \dots, 3$) まわりの角速度ベクトルを $\omega = (\omega_1 \ \omega_2 \ \omega_3)^\top \in \mathbb{R}^3$ とする．ここで，機体の状態ベクトルを $\mathbf{x}_d = (\xi^\top \ \dot{\xi}^\top \ \omega^\top \ \mathbf{q}^\top)^\top \in \mathbb{R}^{13}$ とし，ロータ推力のベクトルを $\mathbf{F} = (F_1 \ F_2 \ F_3 \ F_4)^\top$

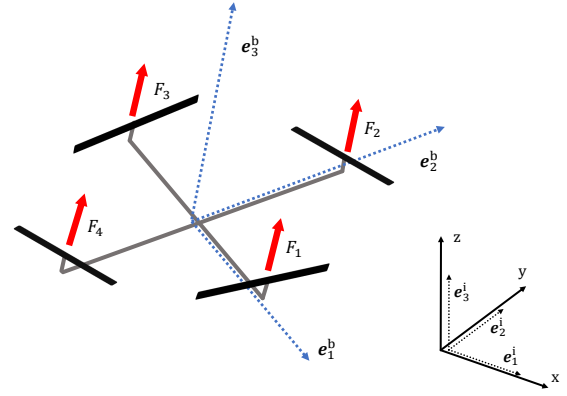


図 1: クワッドコプタの概略図と座標系

とすると，機体の質量 m ，慣性行列 $\mathbf{J} \in \mathbb{R}^{3 \times 3}$ を用いて，クワッドコプタの状態方程式は次式のようにになる．

$$\dot{\mathbf{x}}_d = \begin{pmatrix} \dot{\xi} \\ \frac{1}{m} \mathbf{Q} \mathbf{F}^b - g \mathbf{e}_3^i \\ \mathbf{J}^{-1} (\omega \times \mathbf{J} \omega + \mathbf{T} \mathbf{F}) \\ \frac{1}{2} \Omega \mathbf{q} \end{pmatrix} \quad (2)$$

ただし， g は重力加速度である． $\mathbf{Q}$ と $\mathbf{F}^b$ ， Ω はそれぞれ物体座標系から慣性座標系への変換行列，物体座標系において機体が発生させる推力，クォータニオンの時間微分に関する交代行列である．また， $\mathbf{T}$ は機体の重心から各ロータまでの長さ l とロータ推力と反作用トルクの関係を表す定数 k を用いて次のように表せる．

$$\mathbf{T} = \begin{pmatrix} 0 & -l & 0 & l \\ -l & 0 & l & 0 \\ k & -k & k & -k \end{pmatrix} \quad (3)$$

3 クワッドコプタの推力推定

本節では，クワッドコプタの推力推定の手法について述べる．本研究においては，クワッドコプタの推力と加速度・角加速度についての運動方程式系を最小二乗問題と扱い，リアルタイムでの推力推定を行う．最小二乗法の解法としては，センサによるノイズの影響を軽減する目的で，ガウスニュートン法 [11] を用いる．

クワッドコプタの状態方程式 (2) より， $\mathbf{y} = (\ddot{\xi}^\top \ \dot{\omega}^\top)^\top \in \mathbb{R}^6$ と，ロータの推力 $\mathbf{u} \in \mathbb{R}^4$ について次の方程式を

考える.

$$\begin{aligned} \mathbf{y} &= \mathbf{A}\mathbf{u} + \mathbf{b} \\ &= \begin{pmatrix} Q_1 & Q_1 & Q_1 & Q_1 \\ Q_2 & Q_2 & Q_2 & Q_2 \\ Q_3 & Q_3 & Q_3 & Q_3 \\ 0 & -l/J_{xx} & 0 & l/J_{xx} \\ -l/J_{yy} & 0 & l/J_{yy} & 0 \\ k/J_{zz} & -k/J_{zz} & k/J_{zz} & -k/J_{zz} \end{pmatrix} \mathbf{u} \\ &\quad + \begin{pmatrix} -ge_3^i \\ \mathbf{J}^{-1}\boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega} \end{pmatrix} \\ Q_1 &= 2(q_0q_2 + q_1q_3), \quad Q_2 = 2(q_2q_3 - q_0q_1), \\ Q_3 &= q_0^2 - q_1^2 - q_2^2 + q_3^2 \end{aligned} \quad (4)$$

式 (4) を各時刻で最小二乗問題とみなし、測定値 $\mathbf{y}'(t)$ に対する推力のパラメータ $\hat{\mathbf{u}}(t - \Delta t)$ を推定値として求める (加速度は過去の推力による結果であるため、サンプリング周期 Δt だけ前の推力によって決まるものとする). なお、 $\mathbf{y}'(t)$ における成分のうち、機体の加速度 ξ はクワッドコプタに搭載される加速度センサにより直接測定することが可能である. また、角加速度 $\dot{\boldsymbol{\omega}}$ はジャイロセンサの種類に応じて直接測定可能、または角速度 $\boldsymbol{\omega}$ の疑似数値微分により求めることができる.

式 (4) に対して、ガウスニュートン法を適用すると、次の反復計算によって推力の推定値 $\hat{\mathbf{u}}(t - \Delta t)$ が求まる.

$$\begin{aligned} \mathbf{r}_k &= \mathbf{y}' - (\mathbf{A}\hat{\mathbf{u}}_k + \mathbf{b}) \\ \hat{\mathbf{u}}_{k+1} &= \hat{\mathbf{u}}_k + \alpha(\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{r}_k \end{aligned} \quad (5)$$

ただし、 α は縮小因子であり、 $0 < \alpha \leq 1$ である. 反復計算における初期値は、 $\hat{\mathbf{u}}_0(t - \Delta t) = \hat{\mathbf{u}}(t - 2\Delta t)$ とする. つまり前回の計算時刻での推定結果を参照する.

ここで、式 (4) は $\mathbf{u}$ に関して線形であるため、推力の推定値を各時刻での解析解 $\hat{\mathbf{u}} = (\mathbf{A}^\top \mathbf{A})^{-1} \mathbf{A}^\top \mathbf{y}'$ により反復計算無しで求めることもできる. しかし、この推定値はその時刻のみでの測定値に依存することから、観測ノイズによる誤差の影響を直接的に受け、推定値が大きく発散することがある. 一方、ガウスニュートン法においては、前回時刻における推定値を少しずつ修正することで推定を進めるので、観測ノイズによる誤差の悪影響を抑えることができる.

4 制御系設計

本研究では、クワッドコプタにおける位置と速度、姿勢、角速度の制御を最適制御問題として扱う. その上で、非線形のモデルに対して、各時刻で未来の応答を予測しながら最適制御を行う非線形モデル予測制御 (NMPC) [12] を適用する. 本節ではまず NMPC について簡潔にまとめる. 次に、推力推定の結果を NMPC の制御入力で割

ることで故障診断を行い、その結果を用いて耐故障制御を実現する. NMPC によるクワッドコプタの制御器設計は文献 [13, 14] を参考にした.

4.1 非線形モデル予測制御

まず、NMPC の制御手法について簡単に説明する. 以下の状態方程式で表せる非線形システムを考える.

$$\dot{\mathbf{x}}(t) = \mathbf{f}(\mathbf{x}(t), \mathbf{u}(t), \mathbf{p}(t)) \quad (6)$$

ここで、 $\mathbf{x}(t)$ は状態ベクトル、 $\mathbf{u}(t)$ は制御入力ベクトル、 $\mathbf{p}(t)$ はシステムに含まれる時変パラメータのベクトルである. 拘束条件は次のように与える.

$$\mathbf{C}(\mathbf{x}(t), \mathbf{u}(t), \mathbf{p}(t)) = 0 \quad (7)$$

拘束条件が不等式拘束条件で表される場合でも、ダミー入力を導入することで拘束条件に変換することができる.

以上の状態方程式と拘束条件に対して、有限な評価区間が時間とともに後退する評価関数を考えると、各時刻 t において、以下のように表せる.

$$J = \varphi(\mathbf{x}(t+T), \mathbf{p}(t+T)) + \int_t^{t+T} L(\mathbf{x}(\tau), \mathbf{u}(\tau), \mathbf{p}(\tau)) d\tau \quad (8)$$

ここで、 T は評価区間長さであり、 $\mathbf{x}(\tau)$, $\mathbf{u}(\tau)$ ($t \leq \tau \leq t+T$) は最適制御問題の評価区間上の状態、制御入力ベクトルである. また、関数 φ は終端コスト、関数 L はステージコストと呼ばれる. この評価関数 J を最小にする解は、現在の時刻 t から有限時間未来 $t+T$ までの時間関数 $\mathbf{u}_{\text{opt}}(\tau)$ として与えられる. さらに、実際の時刻 t における制御入力 $\mathbf{u}(t)$ を最適制御入力の初期値 $\mathbf{u}_{\text{opt}}(t)$ によって与えることで、状態フィードバック制御を実現することができる. 以上に述べたフィードバック制御を実現する手法が NMPC と呼ばれる. NMPC を実装するにあたって、各時刻で解く最適制御問題の計算時間はサンプリング周期内で終わる必要がある. 一般にサンプリング周期は、最適制御問題の評価区間よりも大幅に短いため、高速な計算手法が求められる. そこで、NMPC に特化した数値解法として C/GMRES 法 [15] を用いる.

4.2 推力推定と NMPC による故障診断

第 3 節で述べた推力推定の結果を用いて、クワッドコプタの故障診断を行う. まず、制御の拘束条件について述べる. 各ロータの推力には制約があり、 $u_{\min} \leq u_i \leq u_{\max}$ という不等式で与えられる. ただし、 $u_{\min}$ は各ロータの最小推力、 $u_{\max}$ は各ロータの最大推力である. その上でダミー入力 u'_i ($i = 1, \dots, 4$) を導入することで、こ

の不等式拘束条件を等式拘束条件に変換する．等式拘束条件は次のように表せる．

$$C_i = \left(u_i - \frac{u_{\min} + u_{\max}}{2} \right)^2 + u_i^2 - \left(\frac{u_{\max} - u_{\min}}{2} \right)^2 = 0 \quad (i = 1, \dots, 4) \quad (9)$$

NMPC により上記の拘束条件を満たした制御入力を求めることができる．

次に，ロータの故障状態（有効度合）を表すパラメータとして，

$$\mathbf{m}_r = \begin{pmatrix} m_{r1} & m_{r2} & m_{r3} & m_{r4} \end{pmatrix}^\top \in \mathbb{R}^4 \quad (10)$$

を導入する．各ロータの有効度合の推定値 $\hat{\mathbf{m}}_r(t)$ を次のように前回の計算時刻での NMPC による制御入力と推定推力の比として定義することで，故障診断を行う（ Δt は十分小さいため，故障状態の変化は微小といえる）．

$$\hat{m}_{ri}(t) = \frac{\hat{u}_i(t - \Delta t)}{u_{\text{opt},i}(t - \Delta t)} \quad (i = 1, \dots, 4) \quad (11)$$

一般的にマルチコプタのモータを逆回転させることはできないので，ロータが負の推力を出力することはない．また，NMPC の拘束条件により， $u_{\text{opt},i} \geq u_{\min} > 0$ であるので， $\hat{m}_{ri} \geq 0$ である．モータや回路の不具合が原因で，制御入力よりも実際のロータ推力が大きくなる場合は， $\hat{m}_{ri}$ は 1 よりも大きくなる．

4.3 故障診断と耐故障制御の統合

本項では，第 4.2 項で述べた故障診断の結果を NMPC における最適制御問題に反映させることで，アクティブな耐故障制御を構築する．

まず，NMPC で用いる状態方程式について，第 2 節で導入した式 (2) の変更を行う．状態ベクトルは，故障診断の結果を反映させるために $\hat{\mathbf{m}}_r(t)$ を追加して，

$$\mathbf{x}'_d = \begin{pmatrix} \boldsymbol{\xi}^\top & \dot{\boldsymbol{\xi}}^\top & \boldsymbol{\omega}^\top & \mathbf{q}^\top & \hat{\mathbf{m}}_r^\top \end{pmatrix}^\top \in \mathbb{R}^{17} \quad (12)$$

とする．ここで，各ロータにおいて推定される推力は $\hat{u}_i = \hat{m}_{ri}u_i$ ($i = 1, \dots, 4$) となる．制御入力 $\mathbf{u}(t) = \mathbf{F}(t)$ を推定推力 $\hat{\mathbf{u}}$ に置き換えることで，状態方程式は次式のようになる．

$$\dot{\mathbf{x}}'_d = \begin{pmatrix} \dot{\boldsymbol{\xi}} \\ \frac{1}{m}\mathbf{Q} \begin{pmatrix} 0 & 0 & \sum_{i=1}^4 \hat{u}_i \end{pmatrix}^\top - g\mathbf{e}_3 \\ \mathbf{J}^{-1}(\boldsymbol{\omega} \times \mathbf{J}\boldsymbol{\omega} + \mathbf{T}\hat{\mathbf{u}}) \\ \frac{1}{2}\boldsymbol{\Omega}\mathbf{q} \\ 0 \end{pmatrix} \quad (13)$$

式 (13) の状態方程式を NMPC で用いることによって，各ロータの有効度合 $\hat{\mathbf{m}}_r$ を加味した制御入力 $\mathbf{u}_{\text{opt}}$ を計算できる．式 (13) において，故障状態の変化は小さいと仮定し， $\frac{d}{dt}(\hat{\mathbf{m}}_r) = \mathbf{0}$ とした．

次に，フィードバック制御における状態の参照値として， $\mathbf{x}_{\text{ref}} = \begin{pmatrix} \boldsymbol{\xi}_{\text{ref}}^\top & \dot{\boldsymbol{\xi}}_{\text{ref}}^\top & \boldsymbol{\omega}_{\text{ref}}^\top \end{pmatrix}^\top \in \mathbb{R}^9$ と $\mathbf{q}_{\text{ref}} \in \mathbb{R}^4$ ，制御入力の参照値として， $\mathbf{u}_{\text{ref}} \in \mathbb{R}^4$ を定める．その上で，評価関数 (8) における終端コスト φ とステージコスト L を次のように定義する．

$$\varphi(\mathbf{x}) = \sum_{i=1}^9 s_i \left(\mathbf{x}_d - \mathbf{x}_{\text{ref}} \right)_i^2 + \sum_{i=1}^4 s_{i+9} \left(\mathbf{q}^\top \mathbf{q}_{\text{ref}} - \mathbf{q}_{\text{zero}} \right)_i^2 \quad (14)$$

$$L(\mathbf{x}, \mathbf{u}) = \sum_{i=1}^9 s_i \left(\mathbf{x}_d - \mathbf{x}_{\text{ref}} \right)_i^2 + \sum_{i=1}^4 s_{i+9} \left(\mathbf{q}^\top \mathbf{q}_{\text{ref}} - \mathbf{q}_{\text{zero}} \right)_i^2 + \sum_{i=1}^4 r_i \left(\hat{\mathbf{u}} - \mathbf{u}_{\text{ref}} \right)_i^2 - \sum_{i=1}^4 r'_i u'_i \quad (15)$$

ただし， s_i ($i = 1, \dots, 13$) は状態の重み， r_i ($i = 1, \dots, 4$) は制御入力の重み， r'_i ($i = 1, \dots, 4$) はダミー入力の重みである．式 (14) ならびに式 (15) におけるクォータニオンの誤差の取り方は，文献 [16, 17] に基づいており， $\mathbf{q}_{\text{zero}} = \begin{pmatrix} 1 & 0 & 0 & 0 \end{pmatrix}^\top$ である．構成した評価関数においては，事前に与えた状態・入力の参照値と実際の値との誤差が評価区間で小さくなるような，制御入力 $\mathbf{u}_{\text{opt}}(\tau)$ を求めることを目的とする．状態に対する参照値への追従は，終端コスト (14) とステージコスト (15) の第 1 項と第 2 項によって実現される．ステージコスト (15) の第 3 項では，現在の故障状態 $\hat{\mathbf{m}}_r$ において，推定される推力 $\hat{\mathbf{u}}(\tau)$ と目標値 $\mathbf{u}_{\text{ref}}(\tau)$ の誤差が小さくなるように評価関数を構成している．なお，ステージコスト (15) の最終項は，ダミー入力 u'_i の符号を一意に決定し NMPC の計算失敗を回避するために導入する．

次に，NMPC における評価関数の重みを，故障状態に応じて各時刻で変化させることを考える．クワッドコプタの制御において，目標の位置や速度，姿勢，角速度を保つために平衡点で必要な推力を $\mathbf{u}^* \in \mathbb{R}^4$ ($u_{\min} \leq u_i^* \leq u_{\max}$) とする．例えばクワッドコプタの静止（ホバリング）状態を考えると，機体の質量を m ，重力加速度を g として $u_i^* = mg/4$ となる．ここで，全てのロータ $i = 1, \dots, 4$ について， $\hat{m}_{ri}u_{\min} \leq u_i^* \leq \hat{m}_{ri}u_{\max}$ ，つ

まり各ロータの有効度合が

$$\frac{u_i^*}{u_{\max}} \leq \hat{m}_{ri} \leq \frac{u_i^*}{u_{\min}} \quad (16)$$

を満たすとき、クワッドコプタは推力 $\mathbf{u}^*$ を実現可能であると推定され、それにより目標の状態を維持することができる。逆にいずれかのロータにおいて、

$$\hat{m}_{ri} < \frac{u_i^*}{u_{\max}} \quad \text{または} \quad \frac{u_i^*}{u_{\min}} < \hat{m}_{ri} \quad (17)$$

となるとき、クワッドコプタは推力 $\mathbf{u}^*$ を実現することができないと推定できる。このときクワッドコプタが飛行を続けるためには、 $\mathbf{u}^*$ とは異なる推力 $\mathbf{u}_{\text{new}}^*$ によって、新しい平衡状態を作り、飛行を継続する戦略をとる必要がある。新しい平衡状態においては、機体の状態制御の一部を犠牲にする必要があり、位置の制御を続けるためにはヨー方向の回転を許容することが有効である。また、墜落を防止するという観点から、位置の制御においては水平方向よりも高さ（鉛直）方向の追従を優先することを考える（マルチコプタの飛行する場所や条件によっては、水平方向の制御を優先することも考えられる）。

以上の戦略を実現するため、いずれかのロータにおいて不等式 (16) を満たさないとき、評価関数の重み s_i と r_i を通常飛行時とは変化させる。まず、高さ方向の追従を優先するために、 s_3 と s_6 の重みを大きくする。それに加えて、ヨー方向の回転とそれに伴う姿勢変化を許容するため、 s_9 と $s_{10}, \dots, s_{13}$ の重みを小さくする。さらに推力については、故障状態にかかわらず参照値を常に $\mathbf{u}_{\text{ref}} = \mathbf{u}^*$ と定めた上で、不等式 (16) を満たさないときは、推力の重み r_i ($i = 1, \dots, 4$) を小さくする。それにより新しい平衡状態における推力 $\mathbf{u}_{\text{new}}^* \neq \mathbf{u}_{\text{ref}}$ への追従を促進できる。このような戦略によって定められる、故障により推力 $\mathbf{u}^*$ を実現することができないときの評価関数の重みを s_{fi} ($i = 1, \dots, 13$)、 r_{fi} ($i = 1, \dots, 4$) とする。それに対して、機体が正常状態の重みは s_{hi} ($i = 1, \dots, 13$)、 r_{hi} ($i = 1, \dots, 4$) とする。

ここで、 s_{hi} と s_{fi} 、 r_{hi} と r_{fi} を連続的に変化させるために用いる、故障状態を表す新しい変数 $m_{\text{rm}}(t)$ を各時刻で次のように定義する。

$$m_{\text{rm}} = \min_{i=1, \dots, 4} (m_{\text{rmi}}) \quad (18)$$

$$m_{\text{rmi}} = \frac{1}{1 + \exp\left(-\beta_1\left(\hat{m}_{ri} - \frac{u_i^*}{u_{\max}}\right)\right)} - \frac{1}{1 + \exp\left(-\beta_2\left(\hat{m}_{ri} - \frac{u_i^*}{u_{\min}}\right)\right)} \quad (19)$$

上式においては、 $0 < m_{\text{rm}} < 1$ 、 $0 < m_{\text{rmi}} < 1$ を満たす。式 (19) の m_{rmi} はシグモイド関数の重ね合わせで表

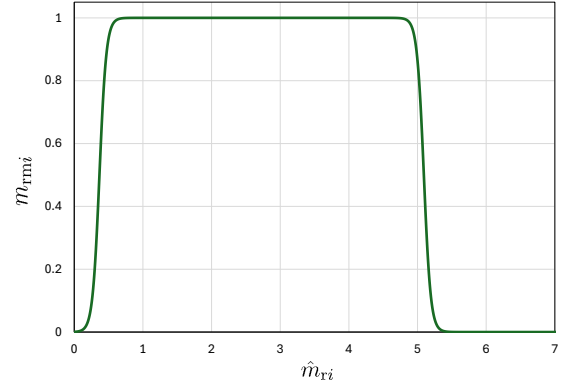


図 2: $\hat{m}_{ri}$ と m_{rmi} の関係

現されており、 $\beta_1, \beta_2 > 0$ はシグモイド関数のゲインである。 β_1 と β_2 を十分大きく取れば、不等式条件 (16) を十分に満たすとき、 $m_{\text{rmi}} \approx 1$ となる。また、不等式条件 (17) を十分に満たすときは、 $m_{\text{rmi}} \approx 0$ となる。不等式の境界においては、 $\hat{m}_{ri} = \frac{u_i^*}{u_{\max}}$ 、 $\frac{u_i^*}{u_{\min}}$ で $m_{\text{rmi}} = 0.5$ であり、境界付近では $0 < m_{\text{rmi}} < 1$ を満たしながら m_{ri} の値に応じて単調かつ連続的に変化する。それにより、NMPC の計算で求められる関数の滑らかさと境界付近での計算の安定性を実現する。例えば、 $u_i^* = 4.9$ 、 $u_{\min} = 0.98$ 、 $u_{\max} = 13$ 、 $\beta_1 = \beta_2 = 20$ のとき、 $\hat{m}_{ri}$ と m_{rmi} の関係は図 2 に示すようになる。式 (18) より、 m_{rm} は m_{rmi} の中での最小値として表される。導入した m_{rm} を用いて、評価関数の重みを各時刻で次のように変化させる。

$$\begin{aligned} s_i &= m_{\text{rm}} s_{hi} + (1 - m_{\text{rm}}) s_{fi} \\ r_i &= m_{\text{rm}} r_{hi} + (1 - m_{\text{rm}}) r_{fi} \end{aligned} \quad (20)$$

式 (20) が表す重みの変化により、上述の戦略を実現することができる。つまり、クワッドコプタが推力 $\mathbf{u}^* = \mathbf{u}_{\text{ref}}$ を余裕をもって実現可能であると故障診断によって判断できる、すなわち全てのロータにおいて不等式 (16) を十分に満たすとき、評価関数の重みは $s_i \approx s_{hi}$ ならびに $r_i \approx r_{hi}$ となる。一方でロータの故障により、 $\mathbf{u}^* = \mathbf{u}_{\text{ref}}$ が実現不可能と判断できるとき、すなわちいずれかのロータが不等式 (16) を満たさないとき、 $s_i \approx s_{fi}$ ならびに $r_i \approx r_{fi}$ となる。

5 シミュレーション

本節においては、第 3 節で述べた推力推定と第 4 節で述べた故障診断、耐故障制御の提案手法が有効であることを、シミュレーションによって確かめる。

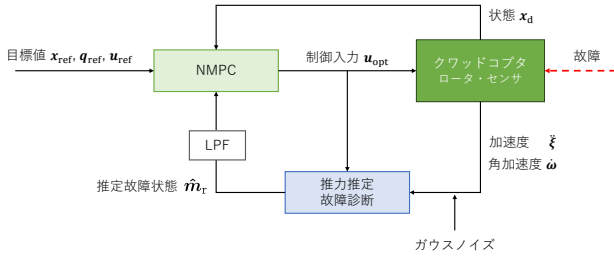


図 3: シミュレーションにおけるシステム構成図

5.1 シミュレーション条件

本研究におけるシミュレーションはすべて汎用のノートパソコン上 (CPU : AMD Ryzen 7 6800U, RAM : 16GB, OS : Windows 11 Home) で, WSL により仮想 Linux 環境を構築し, 実行した. クワッドコプタの制御系に用いる NMPC の実装には, C/GMRES 法を用いる NMPC の自動コード生成ツールである AutoGenU [18,19] を使用した. AutoGenU により生成された C++ ソースファイルを Python インターフェイスにより呼び出すことで, リアルタイムで最適制御入力 u_{opt} を計算することができる. AutoGenU によるシミュレーションと同時に, 第 4 節で述べた推力推定の手法を実行し, 故障診断をリアルタイムで行う. 故障診断と耐故障制御の一連の流れを, 図 3 に示す. ここで推力推定に用いる加速度 $\ddot{\xi}$ と角加速度 $\ddot{\omega}$ に対しては, 実機環境でも推力推定が有効であることを示すために標準偏差 1, 平均 0 のガウスノイズを加える. クワッドコプタの通常の飛行において, 加速度 $\ddot{\xi}$ はたかだか $10^{-1} \sim 10^0$ のオーダー, 角加速度 $\ddot{\omega}$ は 10^0 のオーダーで変化するため, 実機を想定した上で加えるノイズは十分大きいといえる. また, 実際のシステムではノイズ軽減のために, 推力推定の前に $y = (\ddot{\xi}^T \ \ddot{\omega}^T)^T$ に対してローパスフィルタ (Low-Pass Filter: LPF) を使うことが考えられる. しかし, ここでは提案手法におけるノイズ下での有効性を示すために使用しない. 一方, NMPC における計算の安定性を担保するため, $\hat{m}_r$ に対しては 1 次遅れ系のローパスフィルタを使用する.

推力推定におけるガウスニュートン法の縮小因子は $\alpha = 0.25$ とし, 反復回数は 10 回とする. NMPC による耐故障制御について, 評価関数における状態と制御入力の重みをそれぞれ表 1 に示す. またダミー入力の重みは $r'_i = 0.1$ ($i = 1, \dots, 4$) とする. 重みの変化を表す式 (19) におけるシグモイド関数のゲインは $\beta_1 = \beta_2 = 20$ とする. また, NMPC の計算パラメータ値を表 2 に示す. さらにクワッドコプタの機体パラメータを表 3 に

示す. これらの値は Holybro X500 をベースに製作した実機を想定したものである [20].

本稿におけるシミュレーションでは, 位置や姿勢変化の分かりやすさの観点から, 機体はホバリング (静止) 状態を制御目標とする. よって, 制御の参照値としては

$$\begin{aligned} x_{ref} &= (1 \ 1 \ 1 \ 0 \ 0 \ 0 \ 0 \ 0 \ 0)^T \\ q_{ref} &= (1 \ 0 \ 0 \ 0)^T \\ u_{ref} &= (mg/4 \ mg/4 \ mg/4 \ mg/4)^T \end{aligned} \quad (21)$$

を与える. また, 次式に示すように状態と制御入力の初期値は参照値に等しいとする.

$$x_d(0) = (x_{ref}^T \ q_{ref}^T)^T, \quad u(0) = u_{ref} \quad (22)$$

5.2 推力推定を統合した耐故障制御のシミュレーション

推力推定と故障診断を統合した耐故障制御手法の有効性を示すために, 1 枚のロータに故障が発生する場合について, 故障が軽度な場合と重度な場合で耐故障制御のシミュレーションを行う. 比較として, 同じ条件で推力推定・故障診断の結果を利用しない, 通常機体に対しての NMPC による制御のシミュレーションを行う (図 3 において, 故障診断の結果を NMPC に入力しない場合である).

20 秒間のシミュレーションのうち, 1 枚のロータについて, $t = 3$ [s] で有効性を 4 割減少させる. 次に $t = 10$ [s] から 1 秒間かけて, 機体が故障から完全回復して正常となった後に, 今度は $t = 13$ [s] で同じロータを完全に停止させる. なお, 軽度な故障と重度な故障の両方で, 故障発生からその故障の最終状態までの変化には 0.1 秒かかるとする. このとき, 故障診断を利用する耐故障制御と故障診断を利用しない制御を行ったときのシミュレーション結果を図 4 に示す. なお, 故障が始まる時間 $t = 3, 13$ [s] を破線によって示している.

故障診断を利用した耐故障制御では, 故障が軽度な場合とその推移状態ではヨー方向の回転 (主に ω_3 がその回転の角速度を表す) を犠牲にすることなく, 機体の位置を一定に保つことができています. これは, 1 番目のロータの軽度な故障発生とその診断に合わせて, 制御入力 u_1 を大きくしたことで, 目標推力を実現したことによる. 次に, $t = 13$ [s] 以降で 1 枚のロータが完全停止した場面では, ヨー方向の回転を許容することで, 参照位置をほぼ維持できている. 特に高度方向は参照値から ± 1 cm の中で制御されている. ロータが完全停止すると, 目標推力 u_{ref} を実現することができない. そのため, 停止

表 1: NMPC における評価関数の重み

$(s_{h1}, \dots, s_{h13})$	$(80, 80, 80, 0.5, 0.5, 0.5, 0.1, 0.1, 0.1, 0.1, 0.5, 0.5, 0.5)$
$(s_{h1}, \dots, s_{h13})$	$(80, 80, 1000, 0.5, 0.5, 5, 0.02, 0.02, 0.02, 0.02, 0.5, 0.5, 0.05)$
$(r_{h1}, \dots, r_{h4})$	$(0.25, 0.25, 0.25, 0.25)$
$(r_{f1}, \dots, r_{f4})$	$(0.05, 0.05, 0.05, 0.05)$

表 2: NMPC の計算パラメータ

パラメータ	意味	値
t_f	シミュレーション時間	20 [s]
Δt	サンプリング周期	1 [ms]
T_f	最終的な評価区間の長さ	1 [s]
α_{NMPC}	評価区間の長さに 関するパラメータ	2 [s ⁻¹]
k_{max}	GMRES 法の最大反復回数	20
N	評価区間の分割数	50
ζ	C/GMRES 法安定化の ためのパラメータ	1000 [s ⁻¹]

表 3: 機体の物理パラメータ

パラメータ	意味	値
m	機体の質量	2.00 [kg]
g	重力加速度	9.80665 [m/s ²]
u_{max}	ロータの最大推力	13.0 [N]
u_{min}	ロータの最小推力	0.980 [N]
l	機体の重心から 各ロータまでの距離	0.0624 [m]
k	ヨー軸まわりの反作用 トルクと推力の比例係数	0.0500 [m]
J_{xx}	機体のロール軸まわりの 慣性モーメント	0.02167 [kg · m]
J_{yy}	機体のピッチ軸まわりの 慣性モーメント	0.02167 [kg · m]
J_{zz}	機体のヨー軸まわりの 慣性モーメント	0.0400 [kg · m]

したロータの対角側にあるロータの制御入力 u_3 を大きくすることで、1 番目以外のロータで新しい平衡状態を作り出している。一方、故障診断を利用しない通常機体に対する制御について、故障が軽度な場面では機体の位置に揺れ動きがみられ、ヨー方向の回転が生じている。 $t = 13$ [s] 以降、1 枚のロータが完全停止したときは、機体の高度を特に維持できなくなり、約 2m 急降下している。なお、故障診断を利用しない場合でも、軽度な故障において機体がある程度制御できているのは、NMPC が持つロバスト性が理由である。

6 おわりに

本研究では、クワッドコプタの推力推定と故障診断、耐故障制御の新しい手法を提案した。実機を想定したシ

ミュレーションにより、提案手法の有効性を確認できた。今後の展望としては、推力推定や故障診断の提案手法について、先行研究の手法とシミュレーション等を通じて比較することで、その優位性を明らかにしたい。また、推力推定を統合した耐故障制御について、実機を使用した実験でその有効性を確かめたい。

参考文献

- [1] 野波健蔵：続・ドローン産業応用のすべて—進化
する自律飛行が変える未来—, pp. 38–40, オーム社
(2020).
- [2] 野波健蔵：ドローン産業応用のすべて—開発の基礎
から活用の実際まで—, p. 89, オーム社 (2018).
- [3] Freddi, A., Lanzon, A. and Longhi, S.: A feed-
back linearization approach to fault tolerance in
quadrotor vehicles, *IFAC Proceedings Volumes*,
Vol. 44, No. 1, pp. 5413–5418 (2011).
- [4] Nguyen, N. P. and Hong, S. K.: Fault diagno-
sis and fault-tolerant control scheme for quad-
copter UAVs with a total loss of actuator, *En-
ergies*, Vol. 12, No. 6, art. no. 1139 (2019).
- [5] Jung, W. and Bang, H.: Fault and failure toler-
ant model predictive control of quadrotor UAV,
*International Journal of Aeronautical and Space
Sciences*, Vol. 22, pp. 663–675 (2021).
- [6] Nan, F., Sun, S., Foehn, P. and Scaramuzza, D.:
Nonlinear MPC for quadrotor fault-tolerant con-
trol, *IEEE Robotics and Automation Letters*,
Vol. 7, No. 2, pp. 5047–5054 (2022).
- [7] 青木佑介, 浅野雄太, 本田瑛彦, 元岡範純, 大塚敏之:
2 枚のロータが停止したヘキサコプタにおける位置・
姿勢の非線形モデル予測制御, システム制御情報学
会論文誌, Vol. 32, No. 4, pp. 168–176 (2019).
- [8] Puchalski, R. and Giernacki, W.: UAV Fault de-
tection methods, state-of-the-art, *Drones*, Vol. 6,
No. 11, art. no. 330 (2022).

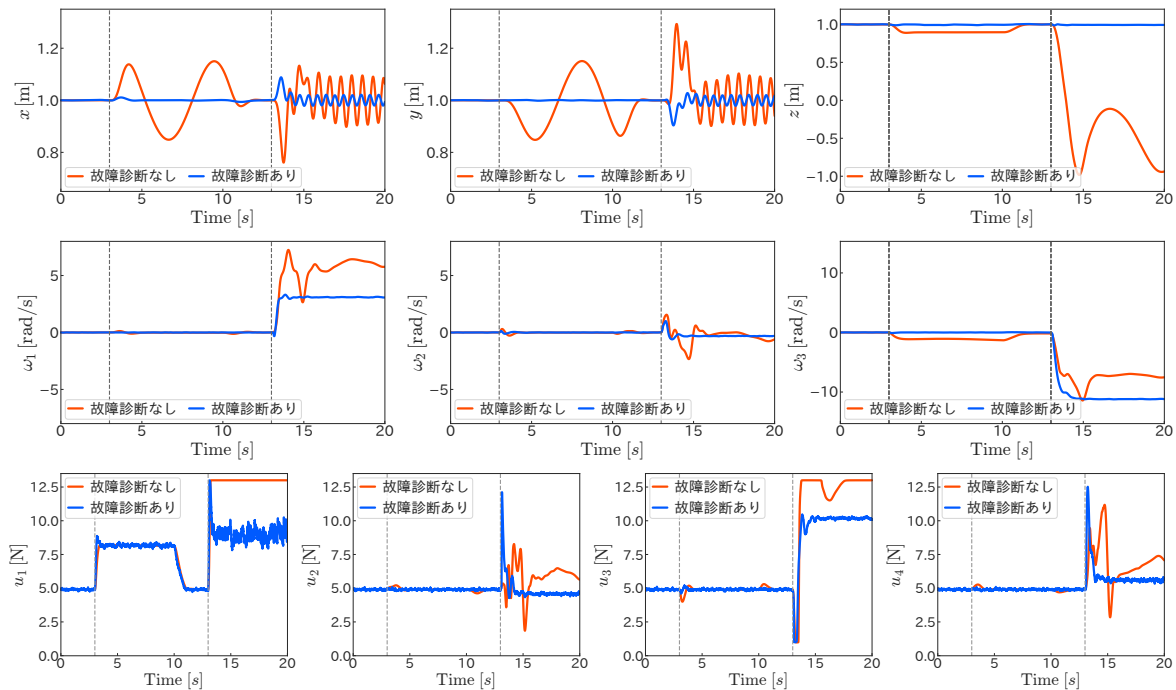


図 4: 故障が軽度な場合と重度な場合での位置と角速度, 制御入力の変化
(1 枚のロータについて $t = 3$ [s] で有効性が 4 割減少, $t = 13$ [s] で完全停止)

- [9] 旭晃弘, 星野健太, 大塚敏之: ドローン 2 機のレースを想定した敵対的経路追従制御, 第 66 回システム制御情報学会研究発表講演会, pp. 870–877 (2022).
- [10] 野波健蔵: ドローン工学入門—モデリングから制御まで—, コロナ社, 306 pp. (2020).
- [11] Boyd, S. and Vandenberghe, L.: *Introduction to Applied Linear Algebra: Vectors, Matrices, and Least Squares*, pp. 381–417, Cambridge University Press (2018).
- [12] 大塚敏之 (編): 実時間最適化による制御の実応用, コロナ社, 243 pp. (2015).
- [13] Kimura, T., Hoshino, K., Asano, Y., Honda, A., Motooka, N. and Ohtsuka, T.: Application of Nonlinear Model Predictive Control to Quadcopter Equipped with Internal Control System, in *2022 SICE International Symposium on Control Systems (SICE ISCS)*, pp. 51–57 (2022).
- [14] Aoki, Y., Asano, Y., Honda, A., Motooka, N., Hoshino, K. and Ohtsuka, T.: Nonlinear model predictive control for hexacopter with failed rotors based on quaternions—simulations and hardware experiments—, *Mechanical Engineering Journal*, Vol. 8, No. 5, paper 21–00204 (2021).
- [15] 大塚敏之: 非線形最適制御入門 (システム制御工学シリーズ), コロナ社, 232 pp. (2011).
- [16] 佐々木貴広, 下村卓: RW を搭載した宇宙機の姿勢制御と偏差 Quaternion に関する研究, システム制御情報学会論文誌, Vol. 28, No. 4, pp. 127–132 (2015).
- [17] 鈴木智, 田原誠, 中澤大輔, 廣谷和之, 野波健蔵: クォータニオンを用いた小型電動ヘリコプタの姿勢制御, 自動制御連合講演会講演論文集, Vol. 51, pp. 226–226 (2008).
- [18] <https://github.com/ohtsukalab/autogenu-jupyter> (2024 年 2 月 6 日閲覧) .
- [19] Katayama, S. and Ohtsuka, T.: Automatic code generation tool for nonlinear model predictive control with Jupyter, in *The 21st IFAC World Congress 2020*, pp. 7033–7040 (2020).
- [20] Holybro X500 + Pixhawk4 Build — PX4 User Guide (Main), https://docs.px4.io/main/en/frames_multicopter/holybro_x500_pixhawk4.html (2024 年 1 月 28 日閲覧) .